

Reliability-Aware Fault Injection for Microscaling-Based DNNs

Project Report

Peeyush Agarwal

CS496 Undergraduate Project, Indian Institute of Technology Kanpur

5 October 2026

Abstract

The adoption of microscaling formats in deep neural network (DNN) accelerators reduces the cost of low-precision inference, but their impact on DNN reliability under hardware faults remains largely unexplored. Unlike conventional floating-point representations, microscaling formats represent multiple values using a shared scale, making the effect of a bit fault dependent not only on the corrupted value but also on the scale and other values within the scaling block. We propose a reliability analysis methodology for microscaling-based DNNs that characterizes how faults in element values and shared scaling factors propagate through the network. The methodology evaluates fault sensitivity across different microscaling formats, block sizes, tensor types, and fault locations, and identifies the representation characteristics that make a model more vulnerable to faults. We conduct an extensive fault-injection study on representative CNN and Transformer models using weight and activation tensors. Our results show that reliability varies substantially across microscaling formats even when they provide comparable inference accuracy, and that faults affecting shared scaling information can have a significantly different impact from faults in individual elements.

Keywords: DNN reliability, microscaling formats, MXFP, block floating-point, fault injection, shared scale, fault sensitivity, low-precision inference.

About this report

The abstract above is the one written for the project brief in August 2026, and I have kept it word for word. Section 1 reproduces the rest of the brief in the same way: the problem, the core hypothesis, the five objectives and the planned study. Everything after that is my account of the work, told in the order it was done, from the first week of September to the start of October.

I have chosen to tell it in order, rather than only presenting the final tables, for one reason. Several of my early conclusions did not survive later experiments, and the way they failed taught me as much as the results that held. Where a result was later revised, I say so at the point it first appears, and Section 5 collects all the revisions in one place.

The results in brief. The core hypothesis holds on every network I tested.

- MX formats with the same accuracy differ in fault vulnerability by up to three orders of magnitude. Measured per inference, the ordering $e3m2 < e4m3 < e5m2$ holds on all eight CIFAR-10 configurations and all nine CIFAR-100 networks tested.
- The main cause is the NaN and infinity codes of the 8-bit formats. A fault that turns an element into NaN corrupts every inference, and on the larger models such faults carry 97–100% of all element damage. Decoding those codes as zero when they are read cuts element damage by at least $29\times$ on those models.
- A fault in the shared scale is always more damaging than a fault in an element, typically by $13\times$ to more than $1000\times$.
- The conventional “any image changed” SDC metric can hide real differences between formats. I replaced it with a per-inference rate, which is stable across evaluation sets.
- Both open design choices in the brief are settled: single-bit injection is sufficient, and exact enumeration over the MX code table beats learned value intervals by two orders of magnitude.

Contents

1	The project brief	2
1.1	Motivation and problem statement	2
1.2	Objectives	3
1.3	Fault model	3
1.4	Study design	3
1.5	Methodology	3
1.6	Expected outcomes	4
2	Background	4
2.1	The element formats	4
2.2	Two fault sites	5
2.3	Metrics	5
2.4	Related work	5
3	The work, in the order it was done	6
3.1	Getting started: scope, hardware and the tool (9–10 September)	6
3.2	First campaigns on ResNet8 (10–11 September)	7
3.3	A first attempt at cheaper campaigns (11 September)	7
3.4	Weights, activations and block size (11–12 September)	7
3.5	A second architecture, and the first mechanism (12–13 September)	8
3.6	More seeds, and a puzzle that exposed the metric (17–20 September)	8
3.7	Moving to the GPU cluster: ground truth, ten seeds and a Transformer (20–22 September)	9
3.8	Chasing the 6-bit puzzle (22–25 September)	10
3.9	Two mistakes caught, and a change of metric (25–27 September)	11
3.10	Settling the brief’s two open design choices (28 September – 1 October)	13
3.11	A read-side guard for special codes (1–2 October)	13
3.12	A bug in my own codec, and a harder dataset (2–3 October)	14
4	Where the objectives stand	15
5	What I got wrong along the way	16
6	Design guidance	17
7	Limitations	17
8	Conclusion	17

1 The project brief

The project started from a working draft titled *Reliability-Aware Fault Injection for Microscaling-Based DNNs: Problem Statement, Fault Model & Study Design*, dated August 2026. Its text is reproduced below without changes, apart from typesetting.

1.1 Motivation and problem statement

Microscaling (MX) formats cut the cost of low-precision inference by having a block of K elements share one scaling factor: the stored value is $v_{i,s} = m_{i,s} \cdot 2^{(X_s - \text{bias})}$, where $m_{i,s}$ is a narrow 4–8-bit element and X_s is an 8-bit scale (E8M0) shared across the block. This coupling changes the fault model fundamentally. In conventional floating-point every value is a self-contained word, so a bit fault perturbs exactly one value. Under microscaling a fault’s effect depends on the corrupted value, on the shared scale, and on the other values in the block — and it can occur at two structurally different sites:

SITE r — ELEMENT

Per-element value. A narrow 4/3/2-bit element m . A fault corrupts a single value.
blast radius $\rightarrow$ 1 value

SITE x — SHARED SCALE

Per-block shared scale. One 8-bit scale shared by K elements. A fault rescales the whole block.
blast radius $\rightarrow$ K values ($\times 2^{\Delta X}$)

Despite the rapid adoption of microscaling in DNN accelerators, its reliability under transient hardware faults is largely unexplored. This project provides a reliability-analysis methodology that characterizes how element and shared-scale faults propagate, and identifies which **representation characteristics** — format, block size, tensor type, and fault location — make a model more or less vulnerable.

CORE HYPOTHESIS

Reliability of microscaling DNNs varies substantially across formats **even at comparable inference accuracy**, and faults in the **shared scaling factor** behave very differently from faults in individual elements — because a single scale fault rescales an entire block of K values at once.

1.2 Objectives

- O1 Fault propagation.** Characterize how single-bit faults in element values vs. shared scaling factors propagate through the network, and quantify the difference in impact.
- O2 Format sensitivity.** Measure fault sensitivity across microscaling formats (MXFP8/6/4 and element variants) and test whether reliability differs even at comparable accuracy.
- O3 Block-size effect.** Determine how block size (scale-sharing granularity) trades inference efficiency against reliability.
- O4 Tensor & location.** Compare sensitivity across tensor types (weights vs. activations) and fault locations (scale bit vs. element bit; MSB vs. LSB; per layer).
- O5 Vulnerability drivers.** Identify the representation characteristics (dynamic range, exponent/mantissa split, scale granularity) that explain observed vulnerability.

1.3 Fault model

We adopt the single-bit-flip model standard in DNN fault-injection studies, generalized to the two microscaling sites, for both weight and activation tensors. Fault **occurrence** (where/how often a bit flips) is modeled separately from fault **impact** (how much the decoded value changes).

Element vs. shared-scale faults.

element flip (bit b of m):	$d = \log v' - \log v $	— 1 value
scale flip (bit b of X):	$v'_{i,s} = m_{i,s} \cdot 2^{(X' - \text{bias})} \quad \forall i$	— all K values
block severity:	$D_s = \sum_i \log v' - \log v = K \cdot X' - X \cdot \log 2$	

A single shared-scale fault rescales the entire block by 2 raised to $(X' - X)$, giving it a blast radius of K — the structural reason scale faults differ from element faults.

Occurrence: per-position rate and multiplicity.

$$\pi_{\text{site},b} = \pi_{\text{site}}^0 \cdot \kappa_{\text{site}}(b), \quad P(S) = \prod_{b \in S} \pi_b \cdot \prod_{b \notin S} (1 - \pi_b), \quad H_{\text{site}} = 1 - \prod_b (1 - \pi_b) \approx \sum_b \pi_b + \sum_{b < b'} \pi_b \pi_{b'} + O(\pi^3)$$

$\kappa(b)$ = per-bit-position susceptibility profile (mean 1); a graded profile reproduces higher rates on more significant bits, $\kappa \equiv 1$ gives the uniform single-bit model. $P(S)$ = Poisson-binomial pattern probability; H = word-corruption rate, whose single-bit term is the baseline and multi-bit tail an optional extension.

1.4 Study design

The study sweeps four representation axes named in the objectives, over representative CNN and Transformer models and both tensor types (Table 1).

1.5 Methodology

Quantization. Quantize model weights and activations to each microscaling configuration; confirm baseline accuracy so reliability is compared at matched accuracy where possible.

Table 1: Study design, as given in the brief.

FORMAT	MXFP8 (E4M3, E5M2), MXFP6 (E3M2, E2M3), MXFP4 (E2M1); optional MXINT8. Shared scale E8M0.
BLOCK SIZE	8 / 16 / 32 / 64 — scale-sharing granularity (fewer values per scale = smaller blast radius).
TENSOR TYPE	Weights (input-independent) and activations (input-dependent).
FAULT LOCATION	Shared-scale bit vs. element bit; MSB → LSB within each field; per-layer position.
MODELS	ResNet8, RepVGG-A0 (CIFAR-10); DeiT-Tiny/Small (ImageNet); optional small LLM.
FAULT MODEL	Single-bit flip at both sites; multi-bit (H-model) as an extension.
METRICS	Fault sensitivity = top-1 change / accuracy drop; per-site failure & SDC rate; per-bit / per-layer maps.

Injection engine. Bit-addressable MXFP representation enabling faults at a chosen bit of either the shared scale or an element, for weights (static) and activations (per input, via hooks); golden-vs-faulty comparison detects top-1 changes and silent data corruption.

Campaign. For each cell of the study matrix, run a statistical fault-injection campaign at a target confidence/error. Value-aware stratified sampling (in the spirit of TreeFI, ICCAD 2026) is used as a supporting tool to keep the large multi-axis campaign tractable; it is not the contribution.

Ground truth. Exhaustive fault injection on a small model (ResNet8) where the microscaling bit space is small enough, providing a reference to validate the statistical estimates.

Analysis. Compare fault sensitivity across the four axes; contrast shared-scale vs. element faults; and correlate vulnerability with representation characteristics to explain the trends.

SUPPORTING TOOL, NOT THE CONTRIBUTION

TreeFI-style value-aware stratified sampling is referenced only to make the multi-axis campaign affordable. The contribution of this work is the **reliability characterization** of microscaling DNNs — not a faster injection method.

1.6 Expected outcomes

- A per-site reliability characterization showing shared-scale faults have a significantly different (typically larger) impact than element faults, consistent with their block-wide blast radius.
- Evidence that reliability varies substantially across microscaling formats even at comparable inference accuracy — so accuracy alone is not a reliability proxy.
- A block-size reliability trade-off: smaller blocks localize scale-fault damage at some storage/efficiency cost.
- Per-tensor and per-location sensitivity maps identifying which bits (scale vs. element; MSB vs. LSB) and which layers most need hardware protection.
- Design guidance: which microscaling configurations are most fault-robust for a given accuracy budget.

Open design choices (settled empirically): learned vs. exact interval characterization for the narrow element formats; single-bit vs. multi-bit (H-model) fault occurrence; and the block-size / format grid actually swept, bounded by available compute.

2 Background

This section collects the definitions the rest of the report relies on.

2.1 The element formats

The scale uses the E8M0 encoding: an unsigned power of two with bias 127, where code 255 is reserved for NaN. Each element is a small sign–exponent–mantissa number. For a nonzero exponent field e it decodes to $(-1)^s(1 + f/2^M)2^{e-\text{bias}}$, where f is the M -bit mantissa field. Table 2 lists the five element formats. The one property that turns out to matter most is in the last column.

The OCP specification chooses the scale so that the largest value in a block lands in the top binade of the element format, $X = \lfloor \log_2 \max_i |v_i| \rfloor - e_{\max}$. Since the largest representable element sits below the top

Table 2: MX element formats. The two 8-bit formats reserve codes for NaN or infinity. The 6- and 4-bit formats do not, so every one of their bit patterns is a finite number.

format	family	sign/exp./mantissa bits	bias	largest value	NaN/Inf codes
e4m3	MXFP8	1/4/3	7	448	2 (NaN)
e5m2	MXFP8	1/5/2	15	57,344	8 (2 Inf, 6 NaN)
e3m2	MXFP6	1/3/2	3	28	none
e2m3	MXFP6	1/2/3	1	7.5	none
e2m1	MXFP4	1/2/1	1	6	none

of that binade, the block maximum can be clipped by up to 12.5% (e4m3, e5m2, e3m2), 6.25% (e2m3) or 25% (e2m1). As a control I also implemented a non-clipping rule, which I call *fit*: the smallest scale at which the block maximum is representable. All results use the OCP rule unless stated otherwise.

2.2 Two fault sites

Figure 1 shows the two sites from the brief. An **element fault** flips one bit of one code: a mantissa bit nudges the value, an exponent bit multiplies it by a power of two, and the sign bit negates it. In e4m3 and e5m2 a flip can also turn the code into NaN or infinity. A **scale fault** flips bit j of X , which multiplies all K values by $2^{\pm 2^j}$ at once, and turns the whole block into NaN if it produces code 255.

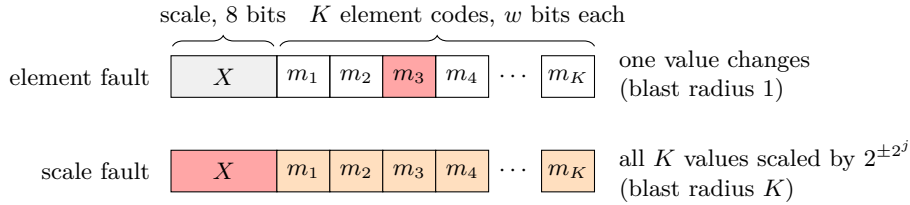


Figure 1: One MX block and its two fault sites. Red marks the flipped word and orange the values it alters.

A short calculation already suggests the scale will matter. A block occupies $8 + Kw$ bits, of which only 8 belong to the scale, about 3% for $K = 32$ and $w = 8$. If every stored bit is equally likely to flip, the expected number of corrupted *values* per block is $8K$ from scale faults and Kw from element faults, a ratio of $8 : w$. So an 8-bit format receives about half of its corrupted values through the 3% of storage that holds scales, and a 4-bit format about two thirds.

2.3 Metrics

Let D be a fixed evaluation set of N images, $\hat{y}_i$ the prediction of the fault-free, MX-quantised network (the *golden* prediction) and $\hat{y}_i^\varphi$ the prediction under fault φ . Two metrics run through the story:

- the **any-image SDC rate**, the fraction of faults that change at least one of the N predictions. This is the conventional metric, and the one I used for the first three weeks;
- the **per-inference rate**, the average over faults of the fraction of the N predictions that change. It answers the question a system designer asks: given one fault, how likely is it that a particular inference is wrong?

Section 3.9 explains why the second replaced the first. I also track the **non-finite rate**, the fraction of faults that produce a NaN or infinity in the output. Proportions carry Wilson 95% intervals [15], and per-inference rates carry normal intervals on the mean over faults.

2.4 Related work

Fault-injection studies have shown that errors in DNNs propagate differently by layer, data type and bit position [3], and tools such as Ares [4] and PyTorchFI [5] made such campaigns routine for FP32 and integer models. Bit-flip attacks [6, 7] show how few flips it can take to destroy accuracy. Statistical fault injection bounds the error of a sampled campaign [8], and TreeFI [9] lowers its cost for FP32 by learning value intervals with similar bit-flip behaviour. None of these model a shared scale, which is the gap the brief identifies. MX formats themselves are described in [1, 2].

3 The work, in the order it was done

3.1 Getting started: scope, hardware and the tool (9–10 September)

Scope. Two drafts existed when I began. One framed the project as a characterisation study, and is the brief reproduced above. The other framed it as a new sampling method, an extension of TreeFI to MX formats. The first decision was to commit to the characterisation. TreeFI stayed in the project only as prior work and as a source of ideas for keeping campaigns affordable, exactly as the brief’s “supporting tool” box says.

Hardware. My machine is a 4-core laptop with no CUDA GPU. That ruled out DeiT on ImageNet locally: one pass over the ImageNet validation set would take about half an hour, and a campaign needs thousands of passes. I decided to start at CIFAR scale with ResNet8 and RepVGG-A0, and to keep the code portable so that heavier runs could move to a GPU later. Before planning anything I measured the real cost of an injection: 47 ms on 100 images on four CPU threads. At that speed the *entire* MXFP8 weight bit space of ResNet8, 638,624 faults, is about a 16-hour run. My first guess had been months, because I had assumed every fault would be scored on the full 10,000-image test set. Knowing that exhaustive ground truth was affordable changed what the project could promise.

The tool. I wrote `mxfi`, a fault-injection library for MX formats. Its core (formats, codec, fault model, sampling, statistics) is plain `numpy` with no deep learning dependency, and a separate layer connects it to PyTorch models. Three design decisions keep the injection faithful to what an accelerator executes:

1. **Deployment form.** Batch normalisation is folded into the preceding convolution, and RepVGG’s branches are fused, before quantisation. An unfolded batch norm would re-normalise whatever a corrupted convolution produced and hide part of the damage.
2. **Code-domain injection.** Tensors are encoded into real MX bit patterns, the fault is applied to the stored code, and only then is the block decoded. This is what lets a scale fault perturb all K values, and lets an element fault land on a NaN code.
3. **Golden-versus-faulty comparison** on a fixed, class-balanced subset of 200 test images, recording for each fault how many predictions changed and whether the output went non-finite.

Algorithm 1 shows a weight campaign. Activation faults go through forward hooks that quantise a layer’s input to MX, flip one bit of one image’s encoding, and decode. Weight faults are persistent: every inference reads the corrupted weight until it is rewritten. Activation faults are transient and affect one inference.

Algorithm 1 Weight fault-injection campaign in `mxfi`

Require: network M , format F , block size K , evaluation set D , budget n

```
1:  $M \leftarrow \text{DEPLOY}(M)$  ▷ fold batch norm, fuse branches
2: for all quantisable layers  $\ell$  do
3:   view  $W_\ell$  as (outputs  $\times$  reduction); split the reduction axis into blocks of  $K$ 
4:    $X_\ell \leftarrow$  per-block scale (OCP rule);  $C_\ell \leftarrow \text{round}_F(W_\ell 2^{-X_\ell})$ 
5:    $W_\ell \leftarrow \text{DECODE}(X_\ell, C_\ell)$  ▷ the network now holds MX values
6: end for
7:  $\hat{y} \leftarrow \arg \max M(D)$  ▷ golden predictions
8: for  $t = 1$  to  $n$  do
9:   draw  $\varphi = (\ell, \text{site}, \text{index}, b)$  uniformly over all stored bits
10:  if site = element then  $C_\ell[\text{index}] \leftarrow C_\ell[\text{index}] \oplus 2^b$ ; re-decode that one weight
11:  else  $X_\ell[\text{index}] \leftarrow X_\ell[\text{index}] \oplus 2^b$ ; re-decode the  $K$  weights of the block
12:  end if
13:   $\hat{y}^\varphi \leftarrow \arg \max M(D)$ ; record which images changed and non-finiteness; restore
14: end for
15: return rates over all faults with confidence intervals
```

Replacing line 9 with an enumeration of every stored bit makes the campaign *exhaustive*: it returns the true failure rate with no sampling error at all.

The first network. ResNet8, the MLPerf Tiny image-classification model [11, 10], trained on CIFAR-10 to 87.01% top-1, which matches the reference. Training took 82 seconds per epoch on the laptop.

3.2 First campaigns on ResNet8 (10–11 September)

The first campaigns were 3000 uniformly sampled weight faults per format at $K = 32$, scored with the any-image metric. Several things showed up at once.

The hypothesis looked right immediately. e5m2 and e3m2 differed in accuracy by 0.05 points but in fault vulnerability by $1.77\times$, with disjoint confidence intervals (Table 3). Accuracy was clearly not a reliability proxy. Interestingly, the 6-bit format looked *worse*. That direction later turned out to be an artefact of the metric (Section 3.9), but the larger point, that equal accuracy does not mean equal reliability, survived everything that came after.

Table 3: The first campaign (ResNet8, CIFAR-10, $K = 32$, 3000 faults each, any-image metric). The direction between e5m2 and e3m2 was later reversed under the per-inference metric.

format	bits	accuracy	any-image SDC [95% CI]	non-finite outputs
e5m2	8	84.80%	28.9% [27.3, 30.6]	7.7%
e3m2	6	84.85%	51.3% [49.5, 53.1]	0.0%
e4m3	8	85.08%	32.3% [30.7, 34.0]	–

The failure modes were different, not just the rates. e5m2 failed rarely but catastrophically: 7.7% of its faults produced NaN or infinity at the output, because it has special codes and a wide exponent. e3m2 never produced a non-finite output, as a format without special codes cannot. This was the first hint of what became the central mechanism of the study.

The severity measure in the brief did not work. The brief scores an element fault by $d = |\log v' - \log v|$. Over 8374 element faults its rank correlation with observed failure was 0.026, effectively zero. The reason is in the definition. Negating a value leaves $|v|$ unchanged, so a sign flip scores zero severity, yet sign flips were the most damaging bit position in the sample. A sampler built on this measure would have given the most dangerous bit almost no budget. Finding a measure that works became a thread I came back to later (Section 3.8).

Scale faults dominated. Scale faults were 2–3 times more likely than element faults to change a prediction, and they changed 37–41 of the 200 predictions on average, against 0.7–15.6 for element faults. Between 9% and 13% of them produced non-finite outputs. But scale bits are only 3.1% of storage, so a uniform campaign of 3000 drew only about 100 of them, and the most important stratum was the least measured.

Two smaller observations. Under the OCP scale rule the 6-bit e2m3 was more accurate than the 8-bit e4m3, and the order flipped back under the non-clipping rule, so the rule itself could distort a format comparison. And accuracy moved by about one point or less across block sizes 8 to 64, which meant block size could be studied as a reliability axis without an accuracy confound.

3.3 A first attempt at cheaper campaigns (11 September)

The shortage of scale samples suggested trying stratified sampling. I compared three allocations at the same budget of 3000: uniform, Neyman allocation over layer and site, and Neyman allocation with an extra weight Φ for blast radius, an idea from the TreeFI-style draft. All three estimators agreed on the model-wide rate to within 0.003, so the weighting was correct. The Φ weight moved 36% of the budget to scale bits and cut the scale-stratum interval by a factor of 10.5 in variance. It also widened the model-wide interval, though, so it was a trade between which estimate is precise, not a free gain. Plain layer-by-site stratification gave $0.94\times$, slightly worse than uniform, because per-layer failure rates were too similar for Neyman allocation to exploit.

The lesson was that the variance does not live across layers. It lives within the value distribution, which is where TreeFI looks. I set sampling aside at this point and returned to it once I had ground truth to test against.

3.4 Weights, activations and block size (11–12 September)

Comparing weights with activations needs care. A weight fault is exposed to all 200 inferences of a campaign, but an activation fault can only ever affect the one inference it lands in. Counting “did any image

change” therefore understates activation faults by about two orders of magnitude. The fair quantity is the chance that a given inference is corrupted, which is the per-inference rate of Section 2.3. I introduced it here for activations only. It took me another two weeks to see that it should replace the any-image metric everywhere. (The activation numbers measured in this phase were also wrong, because of a codec bug I found only at the end of the project; see Section 3.12. The normalisation argument stands.)

Block size is a genuine trade. Raising K from 8 to 64 cuts the share of storage taken by scales from 11.1% to 1.7%, so a scale is less likely to be hit. But each scale fault becomes worse, because it now affects more values (Table 4). The two sites move in opposite directions along this axis, which is why block size cannot be tuned on a single aggregate number.

Table 4: Block size on ResNet8 (e4m3). Larger blocks shrink the scale’s share of storage but make each scale fault worse, while element faults become milder. The last row is per inference, from the later rescored of Section 3.9.

	$K = 8$	$K = 16$	$K = 32$	$K = 64$
scale bits, share of storage	11.1%	5.9%	3.1%	1.7%
scale fault, any-image SDC	0.731	0.890	0.924	0.941
scale fault, predictions changed (of 200)	40.1	45.3	48.7	50.1
element fault, per-inference rate	0.0241	0.0208	0.0159	0.0136

The OCP rule and block size. Injecting one fixed set of element faults at every block size, the any-image rate swung by $2.2\times$ non-monotonically under the OCP rule but stayed flat under the non-clipping rule. I concluded at the time that OCP clipping *manufactures* a block-size trend. Rescored per inference two weeks later, this turned out to be overstated: there is a real, monotone trend under both rules (the last row of Table 4), and the OCP rule only exaggerates it under the any-image metric.

3.5 A second architecture, and the first mechanism (12–13 September)

RepVGG-A0. I trained RepVGG-A0 [12] (7.0M parameters after fusion, 91.44% top-1) and repeated the campaigns. The matched-accuracy result replicated, but its direction reversed: e3m2, which had looked $1.77\times$ worse than e5m2 on ResNet8, was now $7.2\times$ safer. My conclusion at the time was that a format ranking should never be quoted from a single architecture. That warning was sound, but for the wrong reason, as Section 3.9 explains.

Why the failure mode shifts with model size. The non-finite share explained the reversal. On RepVGG, *all 158* of e5m2’s element failures were non-finite events. Not a single ordinary perturbation changed a prediction. On ResNet8 only 28% were. My explanation was that a larger network absorbs ordinary perturbations that a small one cannot, but nothing absorbs a NaN, and only formats with special codes can produce one. So as capacity grows, vulnerability stops depending on how much a flip perturbs a value and starts depending on whether the format can represent NaN at all. This made two predictions: that the effect should be graded by the number of special codes, and that a Transformer, being accurate enough to ignore small perturbations, should show it sharply.

Scale dominance also strengthened with size: the scale-to-element ratio for e4m3 went from $3.0\times$ on ResNet8 to $17.8\times$ on RepVGG, because capacity absorbs a single corrupted weight but not 32 of them at once.

Isolating capacity. RepVGG and ResNet8 differ in depth, topology, training budget and accuracy at once, so to test the capacity explanation properly I trained ResNet8 at five widths (8 to 48) with identical topology, data and a 30-epoch budget. That spans a $35\times$ range of parameters (20k to 692k) and accuracies from 80.0% to 89.6%. On the first seed, the benefit of capacity ranked exactly as predicted, e3m2 most and e5m2 least, and the non-finite rate stayed flat with width while the non-finite share of failures rose. But the curves between the endpoints were badly non-monotone, and each width was one trained model. The clear next step was more seeds.

3.6 More seeds, and a puzzle that exposed the metric (17–20 September)

I wrote a restartable script to retrain every width under further seeds. With two seeds the predicted three-way ordering held in only one of them. The spread between two networks of the same width, differing only in

their seed, was a median $12.8\times$ the uncertainty from the 3000 injections. More injections would not have helped; only more trained networks could. That is a general lesson about fault-injection studies: a result quoted from one trained network is much less certain than its confidence interval suggests.

One cell refused to behave: e4m3 at width 32 was anomalously robust in *both* seeds. To find out why, I injected the same 60 sign flips into the same width-32 network under each format. The flips moved the logits by the same amount in every format (median shift 0.12), yet changed a prediction in 1 of 60 cases under e4m3 and 26 of 60 under e3m2. The difference was in the golden network, not in the faults. The two e4m3 width-32 networks were the only ones of 39 with no evaluation image near a decision boundary (no top-1 versus top-2 margin below 0.2). Every other network had at least one such knife-edge image, which flips under almost any nudge. Across all 39 networks, the any-image rate tracked the number of near-tie images at Spearman $+0.89$.

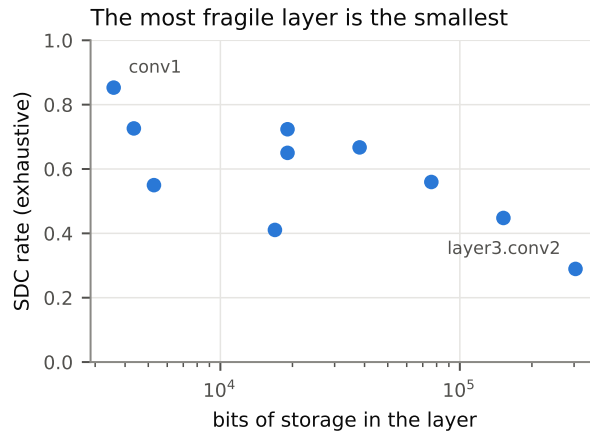
This was the first clear sign that the any-image metric can measure the evaluation set rather than the fault. I did not yet see how far the problem reached. Since the campaign files stored only how many images changed and not which ones, I let the running sweep finish unchanged and changed the campaign code afterwards to record the change set of every fault.

3.7 Moving to the GPU cluster: ground truth, ten seeds and a Transformer (20–22 September)

At this point I got access to an IIT Kanpur server with NVIDIA A100 GPUs. It has no internet access and no copy of ImageNet, which mattered later, but it made three things possible.

Exhaustive ground truth. I flipped every one of the 638,624 bits of a ResNet8’s e4m3 weight storage and scored each one, at about 170 injections per second, in a little over an hour. The true any-image rate was 0.4171. An ordinary 3000-fault campaign had returned 0.4270 with interval $[0.4094, 0.4448]$: within 2.4% of the truth, with the interval covering it. Every sampled number in the study rests on that assumption, and now it was checked rather than assumed. The exact enumeration also showed that 94.4% of all possible scale flips change at least one prediction, and that a scale fault changes 49.9 of 200 predictions on average against 3.3 for an element fault, a $15\times$ difference.

The exact per-layer picture was a surprise (Figure 2). Vulnerability runs *opposite* to layer size. The 3584-bit stem convolution is the most fragile layer, because its errors propagate through the whole network, and the 304,128-bit final convolution is the most robust. A uniform campaign samples layers in proportion to their size, so it spends 48% of its budget on the most robust layer and 0.6% on the most fragile.



A vision Transformer. I trained a small vision Transformer [13] with the width and head count of DeiT-Tiny [14], depth 6 and 4×4 patches (2.7M parameters, 80.4–81.9% on CIFAR-10, three seeds). Every weight matrix is a linear layer, so MX blocks run along the model dimension, and LayerNorm stays unquantised, as in deployment. This gave the cleanest matched-accuracy comparison of the project: e5m2 and e3m2 differed by 0.01 points of accuracy, yet e5m2 was $17.8\times$ more vulnerable on the any-image metric, and 94% of its element failures were non-finite. The prediction from the CNNs held, and more sharply. Neither LayerNorm nor softmax masked the damage, and scale faults dominated here too.

One result did not fit. e3m2 and e2m3 are both 6-bit, both free of special codes and equally accurate, yet differed by $5.8\times$ in element failures. Special codes could not explain that.

3.8 Chasing the 6-bit puzzle (22–25 September)

Ruling things out. The gap was solid (Fisher exact $p = 2.5 \times 10^{-30}$), so I tested explanations one by one. It was not the size of the perturbation: e2m3 actually perturbs weights *less* than e3m2, and within matched perturbation-size bins it still failed $7\times$ more often. It was not which weights were hit, and it was not a difference in margins, since the golden margin distributions were indistinguishable.

The fragility belongs to the network. I then perturbed single weights by a fixed multiple of the layer’s RMS value, a probe with no bit patterns in it at all. At $1.5\times$ RMS, in 1500 trials, it never changed a prediction of the e3m2 or e5m2 networks, but changed the e4m3 network’s in 2–5% of trials. So two quantised networks of equal accuracy could differ by more than an order of magnitude in how much a weight perturbation of a given size disturbs them. Among the 6- and 8-bit formats, the ones with 2 mantissa bits looked $6\text{--}8\times$ safer than the ones with 3, and I first wrote this up as “mantissa width tracks vulnerability”.

A sensitivity measure. To explain it I defined a per-weight sensitivity $s_i = |\partial m / \partial w_i| \cdot \text{rms} / m$, where m is the margin between the top two logits. A perturbation of $\delta \cdot \text{rms}$ should flip a prediction roughly when $s_i \delta > 1$. Computed by backpropagation, s predicted the probe at correlation $+0.998$. Extending the comparison to all five formats then broke my mantissa explanation: e2m1, with the fewest mantissa bits, was the *most* vulnerable. Across formats, sensitivity predicted the perturbation-driven failure rate at $r = +0.93$, while mantissa width correlated at -0.25 . Mantissa width had been a correlate within one group of formats, not a cause.

A severity measure that predicts failure. The same sensitivity gave me a replacement for the brief’s severity measure. For a fault that changes a weight from v to v' ,

$$\text{margin_shift}_i = s_i \frac{|v'_i - v_i|}{\text{rms}}$$

is the fraction of the decision margin the fault uses up. Over 39,337 element faults on the Transformer, the brief’s d had AUC 0.435, slightly worse than chance, relative error reached 0.699, absolute change in RMS units 0.845, and margin_shift reached 0.990 (Figure 3). Its threshold needs no tuning: flagging faults with margin_shift > 1 catches 98.5% of all failures while flagging only 4.4% of the fault space.

Firming up. In the same days I ran a second exhaustive campaign, ResNet8 under e3m2 (483,904 injections), in which not one fault produced a non-finite value, and the sampled estimate again covered the truth (0.1070 against 0.1079). I retrained RepVGG with three seeds, and the ordering e3m2 $<$ e4m3 $<$ e5m2 held on all of them. I also tested value-aware sampling against the two exhaustive campaigns for the first time: splitting the fault space by margin_shift gave a $7\times$ variance reduction on e3m2 and $2.3\times$ on e4m3, for the any-image rate.

What s really measures. One question remained: why does s differ between formats at all? I first guessed that it came from how inputs and gradients line up for particular weights, and tested this with a permutation that destroys that alignment while keeping every value. The sensitivity tail moved by at most 2%, so the guess was wrong. Splitting s into its two factors gave the answer. The gradient factor agrees across all five formats to within $1.08\times$. The whole $10\times$ spread comes from the margin in the denominator, and specifically from the *one* evaluation image closest to a decision boundary. And that image belongs to the evaluation set rather than to the network: $1/\min m$ ranked fifteen networks by failure rate at Spearman $+0.98$ on the campaign’s own 200 images, but only $+0.21$ on 2000 fresh images. This was the near-tie effect from the width sweep again, now explaining a format comparison.

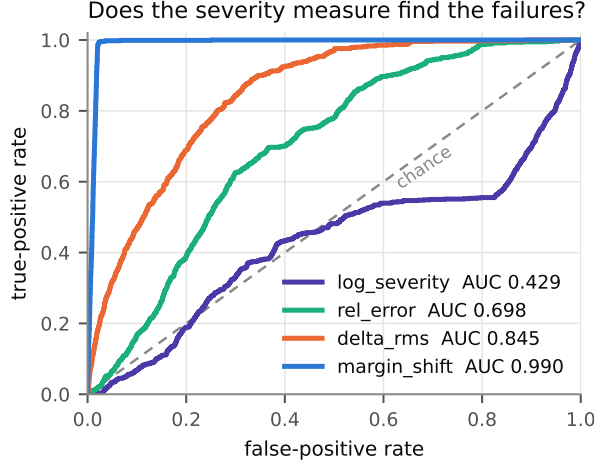


Figure 3: How well each severity measure ranks the faults that actually failed. The measure in the brief falls below the chance diagonal; margin_shift reaches AUC 0.990.

3.9 Two mistakes caught, and a change of metric (25–27 September)

Checking what the campaigns were run on. Training is not bit-reproducible across machines, and the same checkpoint name on my laptop and on the cluster turned out to hold different weights. None of the 24 shared checkpoints matched, while accuracies agreed to a few tenths of a percent, so nothing had looked wrong. I wrote a check that replays a sample of a campaign’s recorded faults against a checkpoint and compares outcomes. It showed that the e4m3 and e3m2 exhaustive campaigns had been recorded on *two different* trained networks (300/300 agreement with one, 202–253/300 with the other). So the “exact” comparison between the two formats was really one network against another, and given the tenfold spread between networks it said nothing about the formats. The first version of the e4m3 sampling result had the same problem. Since then every campaign is tied to the checkpoint that produced it, the replay check runs automatically, and the two exhaustive campaigns are used as two independent references, never as a format comparison.

The metric itself. To redo the comparison properly I replayed one fixed list of 1500 element faults per format against nine disjoint, class-balanced sets of 200 images, on each of two networks. Under the any-image metric, e3m2 and e4m3 looked equal ($1.03\times$ and $0.93\times$), their order reversed from one image set to the next, and the rate moved by up to seven times its own confidence-interval width between sets, tracking each set’s smallest margin (Spearman $+0.80$ to $+0.93$). Scored per inference, the same faults on the same image sets gave $5.6\times$ and $7.6\times$ between the formats, stable to within 1.15 – $1.27\times$ across image sets.

The any-image metric saturates: beyond a few dozen images it mostly reports whether the set contains a near-tie input. The effect was real, and the conventional metric was hiding it. From here on, the per-inference rate is the primary metric of the study.

Rescoring everything. Because every campaign had always recorded the number of changed predictions per fault, all 119 weight campaigns (1.44 million faults on eight model configurations) could be rescored per inference without a single new injection. Table 5 is the result. The ordering $e3m2 < e4m3 < e5m2$ holds on *all eight* configurations per inference, with disjoint intervals, against three of eight under the any-image metric. The “direction reverses across architectures” finding of 12 September was a metric artefact. On the Transformer, a single e5m2 element fault is about 2000 times as likely as an e3m2 one to corrupt a given inference, at 0.01 points of accuracy difference.

Scale dominance survived the rescoring on every model and format, and grew where element faults are rare (Table 6). The capacity result became much sharper (Figure 4): across the $35\times$ range of widths, e3m2 improves $25\times$, e4m3 $1.5\times$, and e5m2 not measurably ($0.96\times$), because 96.9% of e5m2’s element damage is non-finite. The benefit a format draws from a bigger model falls in exact order of its number of special codes. The sensitivity result, alone among the headlines, needed no change: its correlations were identical to three decimals under both metrics.

Value-aware sampling, re-derived. The sampling gains measured earlier were for the any-image rate, so I re-derived them for the per-inference rate. The target changes shape: 89.2% of ResNet8’s e3m2 faults

Table 5: Per-inference rate of element faults, CIFAR-10, $K = 32$, averaged over seeds. Lower is better. The ordering $\text{e3m2} < \text{e4m3} < \text{e5m2}$ holds on every row, with disjoint 95% intervals.

network	seeds	e3m2 (6-bit)	e4m3 (8-bit)	e5m2 (8-bit)
RepVGG-A0	3	0.00001	0.00688	0.05381
ViT	3	0.00003	0.01258	0.06897
ResNet8 (reference)	1	0.00332	0.01699	0.07801
ResNet8, width 8	5	0.01003	0.02005	0.08143
ResNet8, width 16	5	0.00180	0.01374	0.07831
ResNet8, width 24	4	0.00121	0.01210	0.07585
ResNet8, width 32	4	0.00060	0.01354	0.07934
ResNet8, width 48	4	0.00035	0.01302	0.07812

Table 6: Scale-to-element damage ratio, per inference. The direction never reverses. The order- 10^3 figures are lower bounds, because the e3m2 element rate there is statistically indistinguishable from zero.

network	format	scale / element
ResNet8 (exhaustive)	e4m3	$15.3\times$
ResNet8, width 16	e4m3	$13.3\times$
ResNet8, width 16	e5m2	$2.6\times$
ResNet8, width 48	e3m2	$434\times$
ViT	e3m2	order 10^3
RepVGG-A0	e3m2	order 10^3

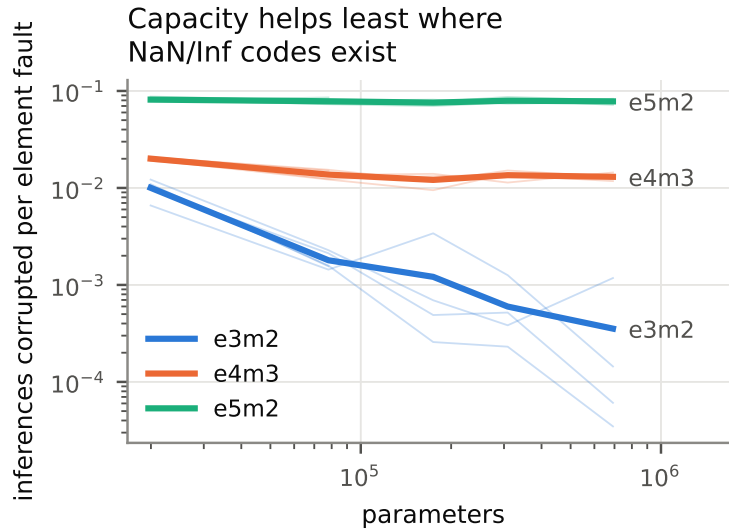


Figure 4: Per-inference rate of element faults against parameter count for ResNet8 at five widths. Each faint line is one trained network; bold lines are means. Capacity helps least for the format with the most NaN/Inf codes, and not at all for e5m2.

corrupt no inference at all, and almost all of the variance sits in a thin tail. The estimator changes too. A per-inference rate is a mean rather than a proportion, so each stratum’s variance has to be estimated from a pilot instead of assumed to be $p(1 - p)$. I scored the whole fault space with `margin_shift`, which costs only gradients, cut it into six strata at its 50th, 75th, 90th, 97th and 99.5th percentiles, spent a fifth of the budget on a pilot, and allocated the rest by Neyman allocation [16]. Replayed against both exhaustive campaigns, the variance fell $57\text{--}120\times$ against uniform sampling on both formats, with no measurable bias. A 3000-injection stratified campaign is as precise as a uniform one of 171,000 (e4m3) to 346,000 (e3m2) injections.

3.10 Settling the brief’s two open design choices (28 September – 1 October)

Single-bit or multi-bit? The brief adopted single-bit flips and left the multi-bit H-model as an extension. To settle it, I sampled 2000 stored words per format on each of two ResNet8 networks and injected *every* single- and double-bit pattern of each word, 390,000 injections in all, recording which predictions changed. A double flip corrupts $1.04\text{--}1.55\times$ as many inferences as a single flip, and an adjacent pair, the pattern a multi-cell upset produces, $0.85\text{--}1.13\times$. The format ordering and scale dominance are unchanged. Double flips are sub-additive, partly because a second flip can move a word *off* the NaN code its partner would have reached, and partly because two scale bits of opposite value cancel. Plugging the measured rates into the H-model’s own expansion, doubles within a word reach 1% of the expected damage only at a per-bit flip probability of $2\text{--}3 \times 10^{-3}$. At that rate ResNet8 already holds more than a thousand flipped bits at once, so the one-fault-per-inference assumption fails through accumulation across words long before multiplicity within a word matters. Single-bit injection is sufficient.

Learned intervals or exact enumeration? The brief also asked whether fault impact should be characterised as TreeFI does, with value intervals learned by a regression tree from a pilot, or by exact enumeration over the MX code table. An MX element alphabet has at most 256 codes, so the decoded change of every (code, bit) pair can be tabulated in closed form at no injection cost. Replayed against both exhaustive campaigns, strata built from the exact $|v' - v|$ alone cut variance $48\text{--}65\times$. Strata learned from a 20% pilot never beat uniform sampling ($0.10\text{--}0.98\times$). Two things defeat the learner. A pilot of a few hundred faults, on a target where 89% of faults do nothing, sees too few failures to place its strata. And in e4m3 the damaging tail is not a range of values at all. It is the discrete set of (code, bit) pairs that land on NaN, which a code table lists exactly. Interval learning solves a problem that FP32 has, an alphabet of 2^{32} values, and that MX does not.

The sign bit, revisited. The exhaustive any-image ranking had made the sign bit the worst element bit, which was the sharpest point against the brief’s severity measure. Rescored per inference (Figure 5) the picture is more interesting. Among faults that leave the output finite, the sign bit is still the worst. In total damage, though, it ranks seventh of eight in e4m3. The 1.27% of element faults that flip a code into NaN each corrupt every inference, and together they carry 78% of all element damage. A sign flip never touches the magnitude bits, so it is the one flip that can never reach the NaN code. The scale bits tell a similar story. On this network the scale bytes lie between 114 and 120, so bits 4–6 are always set and bit 3 almost never. Flipping bit 3 multiplies a block by 256 and corrupts 75% of inferences, and flipping bit 7 overflows it (98%), while flipping bits 4–6 shrinks the block towards zero and corrupts only 3%. Growing a block is catastrophic; shrinking it is mild.

3.11 A read-side guard for special codes (1–2 October)

If most element damage comes from one transition, into a special code, there is an obvious defence: decode any special element code as zero when it is read. A fault that would have produced NaN then only zeroes one weight. I tested this on all three architectures under e4m3 and e5m2. Every element fault that lands on a special code was found by enumeration, so its share of the fault space is exact, and a sample of them was injected with and without the guard. The sampled estimate matched the exhaustive campaign to four significant figures ($0.016357 \rightarrow 0.003749$ against $0.016354 \rightarrow 0.003747$).

The result is in Table 7. On RepVGG and the Transformer, where special-code faults carry 97–100% of element damage, the guard cuts the element per-inference rate by at least $29\times$ and up to $4100\times$, bringing e5m2 down to the level of e3m2. On ResNet8 the reduction is smaller, $3.9\text{--}53\times$, because a small network is also fragile to ordinary perturbations, which the guard does not touch. No guarded fault produced a non-finite output. The guard was simulated in software; in hardware it is one comparator on the read path. It turns “prefer formats without special codes” from a constraint into a choice.

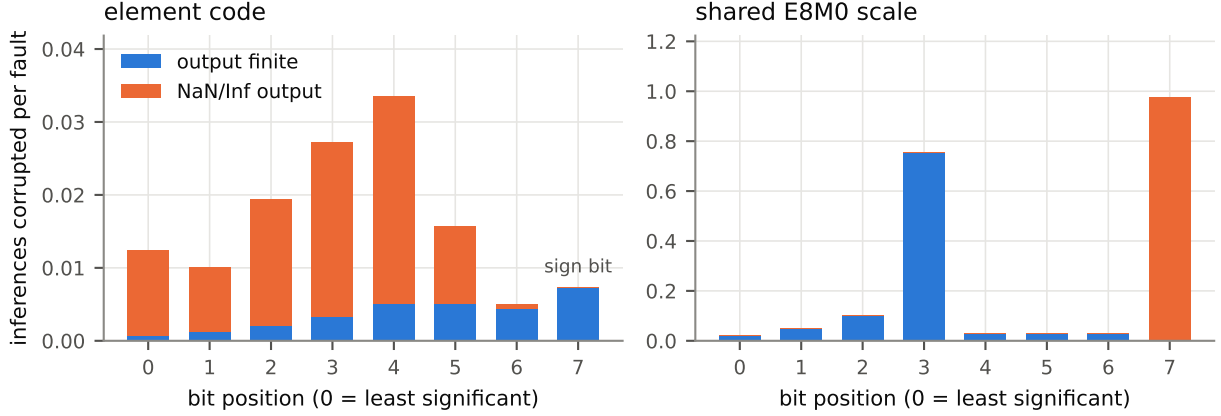


Figure 5: Exhaustive per-bit damage per inference (ResNet8, e4m3, $K = 32$, 638,624 injections), split by whether the output stayed finite. The sign bit does the most finite damage, but most element damage comes from flips into the NaN code, which a sign flip cannot make. Scale bits 3 and 7 are almost never set, so flipping them grows the block $256\times$ or overflows it.

Table 7: Effect of decoding special element codes as zero on read (CIFAR-10). The reduction is the unguarded rate divided by a 95% upper bound on the guarded rate, so it is a lower bound.

network	format	share of element damage from special codes	reduction, at least
ResNet8 (2 nets)	e4m3	78–93%	3.9–10.7×
ResNet8 (2 nets)	e5m2	98–99%	30–53×
RepVGG-A0 (3 seeds)	e4m3	99.5–99.9%	115–389×
RepVGG-A0 (3 seeds)	e5m2	99.9–100%	319–4112×
ViT (3 seeds)	e4m3	97.1–99.5%	29–133×
ViT (3 seeds)	e5m2	99.9–100%	43–2479×

3.12 A bug in my own codec, and a harder dataset (2–3 October)

The codec bug. While preparing the activation experiments for CIFAR-100 I noticed that no activation campaign, on any model, had ever recorded a non-finite output, even though enumeration showed that activations have the same share of special-code faults as weights. The cause was my codec: it mapped a NaN input to zero. An activation-quantised layer re-encodes its input, so a NaN created by an activation fault was quietly erased at the next layer. The OCP conversion does the opposite. It takes the shared scale from the block maximum, so a NaN anywhere in a block makes the whole block NaN. I fixed the codec to do exactly that, added a test that pins the behaviour, and reran every activation campaign on 100 images with 4000 faults each. Weight campaigns were unaffected, since trained weights contain no NaN to re-encode.

The corrected numbers (Table 8) withdraw my earlier claim that activation faults barely matter on the larger models. Per inference, an element fault in a weight does $0.6\text{--}2.2\times$ the damage of one in an activation, and under e4m3 the two are statistically indistinguishable on CIFAR-10. The reason is the same in both tensors: about 1% of faults land on a special code and destroy the inference. The real difference between weights and activations is *exposure*: a weight fault persists across every inference until the memory is rewritten, while an activation fault lives for one. Scale faults dominate in both, $14\times$ in activations and $11\times$ in weights on ResNet8.

Table 8: Per-inference rate of element faults in weights and activations, after the codec fix (CIFAR-10, $K = 32$). Activation intervals are bootstrapped over 100 images.

network	format	weight	activation [95% CI]	weight / activation
ResNet8	e4m3	0.0170	0.0122 [0.0077, 0.0182]	1.4×
RepVGG-A0	e4m3	0.0048	0.0075 [0.0049, 0.0101]	0.64×
ViT	e4m3	0.0122	0.0114 [0.0080, 0.0150]	1.07×
ResNet8	e5m2	0.0780	0.0359 [0.0302, 0.0421]	2.2×
RepVGG-A0	e5m2	0.0549	0.0378 [0.0319, 0.0442]	1.45×
ViT	e5m2	0.0659	0.0486 [0.0420, 0.0554]	1.36×

CIFAR-100 in place of ImageNet. The brief’s Transformer study was DeiT on ImageNet, and ImageNet was not available on any machine I could use. CIFAR-100 was the nearest affordable substitute: ten times the classes and a tenth of the images per class, so networks sit much closer to their decision boundaries, which is exactly the property the margin results had shown to matter. I retrained all three architectures with unchanged recipes, three seeds each (ResNet8 58%, RepVGG-A0 70–71%, ViT 47–49%).

Every headline replicated. Formats were matched in accuracy (four formats within 0.01 points on the ViT), the ordering $e3m2 < e4m3 < e5m2$ held on all nine networks, scale faults outweighed element faults everywhere ($1.7\times$ to $3600\times$), and NaN codes carried 94–100% of $e4m3$ and $e5m2$ element damage on the larger models.

What changed was the floor (Table 9). With smaller margins, the formats without special codes became $3\text{--}22\times$ more vulnerable, while $e4m3$ and $e5m2$ barely moved, since a NaN corrupts an inference however wide the margins are. The gap between formats therefore narrows: on the ViT the $e4m3/e3m2$ ratio falls from $461\times$ to $19\times$. The advantage of NaN-free formats is still large, but it is smallest on exactly the harder tasks that real deployments face. The read-side guard still works, with reductions of at least $13\text{--}1387\times$ on RepVGG and the ViT. Among the NaN-free formats the ordering does not carry over: on CIFAR-100 the ViT ranks $e2m1$ as the safest of the three, which is what the sensitivity result predicts, since perturbation damage follows the network and not the field widths.

Table 9: Per-inference rate of element faults, CIFAR-10 $\rightarrow$ CIFAR-100, mean over seeds.

network	e3m2 (no special codes)	e4m3	e5m2	e4m3/e3m2 gap
ViT	$2.7e-5 \rightarrow 6.1e-4$ ($22\times$)	$0.0126 \rightarrow 0.0114$	$0.069 \rightarrow 0.077$	$461\times \rightarrow 19\times$
RepVGG-A0	$1.4e-5 \rightarrow 5.6e-5$ ($3.9\times$)	$0.0069 \rightarrow 0.0081$	$0.054 \rightarrow 0.057$	$476\times \rightarrow 144\times$
ResNet8	$1.8e-3 \rightarrow 5.7e-3$ ($3.2\times$)	$0.0137 \rightarrow 0.0169$	$0.078 \rightarrow 0.082$	$7.6\times \rightarrow 3.0\times$

Wrapping up. The last runs added seeds to ResNet8 on CIFAR-100 and repeated the activation campaigns under $e5m2$ on all six models. On 3 October I pushed the library, every experiment script, the trained networks and the raw campaign outputs to the project repository.

In total the study recorded about 1.44 million sampled weight faults in 119 campaigns on CIFAR-10, two exhaustive campaigns (1.12 million injections), 390,000 multi-bit injections, and further campaigns for activations, the NaN guard and CIFAR-100, on more than sixty independently trained networks.

4 Where the objectives stand

Table 10 answers each objective in the brief, and Table 11 sets the expected outcomes against what was found.

Table 10: The five objectives and their answers. All rates are per inference.

objective	answer
O1 Fault propagation	A scale fault is always more damaging than an element fault, typically $13\times$ to over $1000\times$, on every architecture, format, tensor type and dataset. Less only for $e5m2$ ($2.6\times$), whose element faults are already NaN-dominated. Scale faults also produce non-finite outputs in 7–13% of cases for every format, since E8M0 has its own NaN code and its top bit is worth 128 exponent steps.
O2 Format sensitivity	Formats of equal accuracy differ by up to three orders of magnitude. $e3m2 < e4m3 < e5m2$ on all eight CIFAR-10 configurations and all nine CIFAR-100 networks.
O3 Block size	Larger blocks shrink the scale’s share of storage ($11.1\% \rightarrow 1.7\%$) but make each scale fault worse, while element faults become milder. A genuine trade; with protected scales, large blocks are safe.
O4 Tensor & location	Per fault, weights and activations are about equally dangerous ($0.3\text{--}2.2\times$); they differ in exposure. Among finite faults the sign bit is the worst element bit, but flips into NaN dominate total damage. In the scale, flips that grow the block are catastrophic and flips that shrink it are mild. The small stem layer is the most fragile.
O5 Vulnerability drivers	First, NaN/Inf codes: they carry 97–100% of element damage on the larger models, and capacity cannot absorb them. Second, for the remaining perturbation damage, the network’s margin-normalised sensitivity ($r = +0.93$), not mantissa or exponent width.

Table 11: The brief’s expected outcomes against what was found.

expected outcome	what was found
Shared-scale faults have a significantly different, typically larger, impact	Confirmed everywhere; never reversed on any model, format or dataset.
Reliability varies across formats at comparable accuracy	Confirmed, by up to three orders of magnitude; explained by special codes and network sensitivity.
A block-size trade-off	Confirmed, with the two fault sites moving in opposite directions.
Per-tensor and per-location maps	Exhaustive per-bit and per-layer maps for ResNet8; weight/activation comparison on six models.
Design guidance	Section 6, including a read-side guard the brief did not anticipate.
Open choice: learned vs. exact intervals	Settled: exact enumeration, $48\text{--}65\times$ against $\leq 1\times$.
Open choice: single vs. multi-bit	Settled: single-bit is sufficient.
Open choice: the grid actually swept	All five formats and all four block sizes on CIFAR; DeiT/ImageNet replaced by a ViT on CIFAR-100 (Section 7).

5 What I got wrong along the way

Table 12 lists every conclusion I later had to revise or withdraw. I include it because the corrections changed the final picture, and because most of them share a small number of causes.

Table 12: Conclusions that were later revised or withdrawn.

first conclusion	what replaced it	cause
e3m2 is $1.77\times$ less reliable than e5m2 on ResNet8	e3m2 is safer, on every network	any-image metric
Format ranking reverses between architectures	The ranking is the same everywhere	any-image metric
OCP clipping reverses format accuracy rankings	Did not replicate on RepVGG; clipping costs accuracy, severely for MXFP4	single model
OCP clipping fabricates the block-size trend	A real trend exists under both scale rules; OCP only exaggerates it	any-image metric
The strict three-way capacity ordering (one seed)	Unresolved at 2–3 seeds, established at 10	too few seeds
Mantissa width drives perturbation vulnerability	Network sensitivity does; e2m1 breaks the mantissa story	too few formats
The sensitivity gap comes from alignment	It comes from the margin of the hardest evaluation image	reasoning by elimination
Exhaustive e3m2 vs. e4m3 confirms a $5\times$ format gap	The two campaigns came from different networks	checkpoint mismatch
Value-aware sampling gives no gain on e4m3	$2.3\times$ any-image, $57\times$ per inference	checkpoint mismatch
Sign bit is the most damaging bit	Only among finite faults; NaN flips dominate in e4m3	any-image metric
Activation faults barely matter on larger models	Per fault, about as damaging as weight faults	codec bug

Three lessons stand out, and I would apply them to any fault-injection study:

1. **Score per inference.** The any-image metric depends on whether the evaluation set happens to contain a near-tie input, and it hid or reversed real effects four times.
2. **Replicate over trained networks, not only over injections.** The spread between networks of the same architecture was about ten times the uncertainty from 3000 injections.
3. **Tie every campaign to its exact checkpoint, and check the plumbing.** A replay check would have caught the checkpoint mismatch at once, and an absent non-finite output where one was expected is what exposed the codec bug.

6 Design guidance

For designers of MX accelerators:

1. **Protect the shared scale first.** A scale fault typically does $13\times$ to more than $1000\times$ the damage of an element fault, and scales are only 2–11% of storage. Parity or ECC on scale bytes is the cheapest protection with the largest payoff. If only some scale bits can be protected, protect those normally at 0, whose flips grow the block.
2. **Neutralise special codes.** Prefer element formats without NaN/Inf codes, or decode special element codes as zero on read. The guard removes most of the element vulnerability of e4m3 and e5m2 on the larger models.
3. **Do not judge reliability by accuracy or bit width.** Formats of equal accuracy differ by orders of magnitude, and among NaN-free formats the ranking depends on the network and task.
4. **Treat block size as a trade-off.** Larger blocks save scale storage but make each scale fault worse. With protected scales, large blocks are safe.
5. **Prioritise early layers.** The small stem layer is the most fragile; protection should follow vulnerability, not storage size.

For anyone running fault-injection campaigns on MX models:

6. Report the per-inference rate, not the any-image SDC rate.
7. Inject single-bit faults; double-bit faults add no first-order information.
8. Stratify by exact per-code enumeration of the decoded change, weighted by gradient sensitivity where available, for $57\text{--}120\times$ fewer injections at the same precision.
9. Tie every campaign to its exact checkpoint, and verify it by replaying recorded faults.

7 Limitations

ImageNet and DeiT were not evaluated. ImageNet was not reachable from any machine available to me, so CIFAR-100 stood in for it, with a small vision Transformer trained from scratch at DeiT-Tiny width. That network reaches only 47–49% on CIFAR-100, far below a pretrained DeiT on ImageNet. The CIFAR-100 study shows that the conclusions survive a harder task with smaller margins; it is not a measurement of DeiT on ImageNet. The trend it reveals, a narrowing gap between formats as margins shrink, suggests that absolute gaps at ImageNet scale may be smaller than on CIFAR-10. The directions of the effects, which held on every network, are the more transferable result.

Model scale. The largest network has 7 million parameters. The optional small LLM and MXINT8 were not studied. The capacity results suggest that larger models absorb perturbations but not NaNs, which would make special codes even more dominant, but I did not test this directly.

Fault model. Only faults in stored values are modelled: weights, activations and their scales. Datapath, accumulator, control and timing faults are out of scope. Every stored bit is assumed equally likely to flip ($\kappa \equiv 1$ in the brief’s notation); a graded profile would reweight the per-bit results but not change them.

Other. Campaigns score a fixed subset of 200 images (100 for activations) to keep millions of injections affordable. The per-inference rate is robust to this choice, but absolute rates carry the uncertainty of the subset. The NaN guard was evaluated in software, not in hardware, and the mechanism behind the element-side block-size trend is not established.

8 Conclusion

The brief asked whether the reliability of microscaling DNNs varies across formats at comparable accuracy, and whether faults in the shared scale behave differently from faults in elements. Both answers are yes, on every network I tested. Formats of equal accuracy differ by up to three orders of magnitude, and a scale fault is always more damaging than an element fault, typically by $13\times$ to more than $1000\times$.

The study also found out why. Most element damage comes from the small fraction of flips that land on NaN or infinity codes. No amount of model capacity absorbs them, and a simple read-side guard removes

them. What is left is governed by the decision margins of the network, not by how many mantissa bits the format has.

Along the way the project produced a bit-accurate fault-injection library validated against exhaustive ground truth, showed that the conventional SDC metric can hide the differences it is meant to reveal, replaced the brief’s severity measure with one that predicts failure and cuts campaign cost by two orders of magnitude, and settled both of the brief’s open design choices. The most useful next step is to repeat the key campaigns on a pretrained Transformer at ImageNet scale, which this project could not reach.

Code and data. The `mxfi` library, every experiment script, the trained networks and the raw campaign outputs are in the project repository (https://github.com/peeyushagarwal2004/CS496_UGP). A detailed log of all 72 findings, including those later revised or withdrawn, is in `docs/findings.md`.

Acknowledgements. I am grateful to my supervisor for the project brief and for guidance throughout, and to IIT Kanpur for access to the GPU server on which most of the later experiments ran.

References

- [1] B. D. Rouhani et al., “Microscaling data formats for deep learning,” *arXiv:2310.10537*, 2023.
- [2] Open Compute Project, “OCP Microscaling Formats (MX) Specification, Version 1.0,” 2023.
- [3] G. Li, S. K. S. Hari, M. Sullivan, T. Tsai, K. Pattabiraman, J. Emer, and S. W. Keckler, “Understanding error propagation in deep learning neural network (DNN) accelerators and applications,” in *Proc. SC*, 2017.
- [4] B. Reagen et al., “Ares: A framework for quantifying the resilience of deep neural networks,” in *Proc. DAC*, 2018.
- [5] A. Mahmoud et al., “PyTorchFI: A runtime perturbation tool for DNNs,” in *Proc. DSN Workshops*, 2020.
- [6] S. Hong, P. Frigo, Y. Kaya, C. Giuffrida, and T. Dumitras, “Terminal brain damage: Exposing the graceless degradation in deep neural networks under hardware fault attacks,” in *Proc. USENIX Security*, 2019.
- [7] A. S. Rakin, Z. He, and D. Fan, “Bit-flip attack: Crushing neural network with progressive bit search,” in *Proc. ICCV*, 2019.
- [8] R. Leveugle, A. Calvez, P. Maistri, and P. Vanhauwaert, “Statistical fault injection: Quantified error and confidence,” in *Proc. DATE*, 2009.
- [9] TreeFI: value-aware statistical fault injection for DNNs, in *Proc. ICCAD*, 2026.
- [10] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proc. CVPR*, 2016.
- [11] C. Banbury et al., “MLPerf Tiny benchmark,” in *Proc. NeurIPS Datasets and Benchmarks Track*, 2021.
- [12] X. Ding, X. Zhang, N. Ma, J. Han, G. Ding, and J. Sun, “RepVGG: Making VGG-style ConvNets great again,” in *Proc. CVPR*, 2021.
- [13] A. Dosovitskiy et al., “An image is worth 16x16 words: Transformers for image recognition at scale,” in *Proc. ICLR*, 2021.
- [14] H. Touvron, M. Cord, M. Douze, F. Massa, A. Sablayrolles, and H. Jégou, “Training data-efficient image transformers & distillation through attention,” in *Proc. ICML*, 2021.
- [15] E. B. Wilson, “Probable inference, the law of succession, and statistical inference,” *J. American Statistical Association*, vol. 22, 1927.
- [16] J. Neyman, “On the two different aspects of the representative method,” *J. Royal Statistical Society*, vol. 97, 1934.